Project Progress Report: Development and Wireless Control of an Unmanned Ground Vehicle (Part 1)

K manvit rithwik reddy & Hari shiva rama krishna

02-11-2026

1 Introduction & Objectives

The primary objective of this project is to build and control a two-wheel differential drive Unmanned Ground Vehicle (UGV) using an ESP32 microcontroller development board and an L293D motor driver integrated circuit. The drive chassis features two DC gear motors coupled to drive wheels and a passive omnidirectional caster wheel for ground balance.

The key goals for Part 1 of this project include:

- Interfacing the ESP32 microcontroller with the L293D dual H-bridge motor driver to govern speed and direction using Pulse Width Modulation (PWM).
- Implementing a dual-power architecture that isolates logic power supply from high-current inductive motor draw to prevent micro-controller reset loops and brownout conditions.
- Establishing a stable wireless Bluetooth Low Energy (BLE) communication link between an Android smartphone and the ESP32 using the Dabble mobile controller application.
- Calibrating motor direction polarities, zero-radius turn dynamics, and deadband thresholds for seamless remote operation.

2 Day 1

We officially commenced the UGV project on 29-10-2026. We collected the allotted hardware kit from the laboratory. The chassis frame was already pre-assembled with both TT gear DC motors, drive tyres, and the omnidirectional caster wheel firmly secured.

Before proceeding to circuit assembly, we focused on verifying the mechanical integrity and rotation of the tyres and motor gearboxes.

1. **Tyres and Drive Wheel Inspection:** We inspected the mounting of both rubber tyres on their plastic wheel rims to ensure concentric alignment. The wheels were press-fitted onto the dual-flat shafts of the TT gear motors. We checked for any wobble or lateral play along the axles.
2. **Motor Gearbox Free-Run Verification:** We applied a direct test voltage across the terminals of each TT motor to verify wheel rotation. Both the left and right drive wheels spun smoothly in forward and reverse directions without mechanical binding or excessive gear friction.
3. **Caster Wheel Rotation:** We inspected the omnidirectional caster wheel mounted on the chassis. The pivot swivel and roller bearing were checked to ensure clean 360-degree rotation without friction on smooth surfaces.

No other active sensors or electrical components were tested during this phase, as instructed.

2.1 end_of_the_day

1. Verified that the pre-mounted chassis, tyres, TT gear motors, and caster wheel were mechanically sound.

2. Completed the physical inspection and functional rotation check of both drive tyres.

3 Day 2

On the second day, 30-10-2026, we began connecting our components using the breadboard, jumper wires, ESP32 development board, and the L293D motor driver IC.

Initial Circuit Assembly and USB Power Setup

We initially configured the circuit to run entirely from the laptop USB port without connecting an external battery:

- The ESP32 ground and L293D ground pins were linked to form a single common ground bus.
- The +5 V (VIN) output from the ESP32 development board was routed to supply both the L293D internal logic rail (V_{CC1}) and motor supply rail (V_{CC2}).
- The L293D enable pins (Pin 1 for Motor 1 and Pin 9 for Motor 2) were held permanently HIGH to enable both H-bridges.
- Four logic control outputs from the ESP32 (GPIOs 18, 19, 21, and 22) were connected directly to the L293D phase input pins (1A, 2A, 3A, and 4A).
- The leads from the left and right TT gear motors were wired to the L293D drive outputs.

Hardware Verification Without Firmware

Once the wiring was finished, the motors remained completely stationary upon plugging in the USB cable. We realized that without an uploaded program, the ESP32 GPIOs remained in a high-impedance floating state.

To confirm that the wiring and driver IC were functioning correctly, we temporarily pulled the jumper wire connected to L293D Pin 2 (1A) and touched it directly to the +5 V power rail. The left motor immediately began spinning continuously, confirming that the power distribution, IC, and motor terminals were properly wired.

3.1 end_of_the_day

1. Completed point-to-point breadboard connections between the ESP32, L293D driver IC, and motor terminals.

2. Verified hardware circuit paths through manual high-level logic stimulation.

4 Day 3

On 31-10-2026, we proceeded to flash firmware from the Arduino IDE to test autonomous motor sequencing and PWM speed control. However, we encountered significant communication and driver hurdles.

Flashing Failures and Port Identification

When attempting to upload a basic test sketch, the Arduino IDE console printed:

```
Serial port COM3: Connecting...
A serial exception error occurred: Write timeout
Failed uploading: uploading error: exit status 1
```

We investigated the issue through the following debugging steps:

- **Manual Boot Sequencing:** We attempted holding down the physical `BOOT` button on the ESP32 while pressing upload, but the write timeout persisted.
- **Device Manager Analysis:** Opening the Windows Device Manager revealed a yellow warning triangle under *Other Devices* on **CP2102 USB to UART Bridge Controller**. The operating system lacked the proper Virtual COM Port (VCP) driver. Furthermore, the selected COM3 port in the IDE was actually a virtual Bluetooth serial link, not the USB hardware interface.
- **Driver Installation:** We downloaded and installed the official Silicon Labs CP210x VCP driver. Upon installation, the warning disappeared, and a recognized serial port was assigned under *Ports (COM & LPT)*.
- **Upload Speed Adjustment:** We lowered the IDE upload speed to 115200 baud to ensure stable packet transmission over the data cable. Flashing was then completely successful.

Power Rail Instability and Battery Integration

With the code flashed, both motors began to draw power simultaneously from the laptop USB bus. The high inductive current draw caused the ESP32 to brown out and reboot intermittently.

To solve this, we introduced our 2S 18650 Li-ion battery pack (7.4V–8.4V) into the circuit:

- The 2S battery positive lead was connected directly to L293D Pin 8 (V_{CC2}), dedicating high voltage exclusively to motor drive power.
- The battery negative lead was tied to the common ground rail.
- The ESP32 and L293D logic supply (V_{CC1}) remained powered by a dedicated 5V USB power bank connection.

4.1 end_of_the_day

1. Resolved the serial write timeout error by installing the CP2102 USB-to-UART bridge driver and selecting the correct COM port.
2. Separated motor drive power from digital logic using a 2S battery pack and power bank dual-supply architecture.

5 Day 4

On 01-11-2026, we worked on wireless Bluetooth Low Energy (BLE) control using the Dabble mobile application.

API Migration and Bluetooth Pairing

When compiling our BLE sketch, the compiler generated fatal errors:

```
error: 'ledcSetup' was not declared in this scope
error: 'ledcAttachPin' was not declared in this scope; did you mean 'ledcAttach'?
```

This error occurred because our Arduino environment was running ESP32 Core version 3.x, where the legacy PWM functions were replaced by the unified `ledcAttach(pin, freq, resolution)` API. We refactored our code to comply with the Core 3.x standards.

Next, during mobile pairing, the Dabble app repeatedly showed an immediate "Disconnected" status. We resolved this by enabling runtime location and nearby devices permissions on our Android phone and clearing old BLE cache entries.

Directional Calibration and Steering Inversion

Once connected, the directional controls exhibited motion discrepancies:

- **Forward/Backward Inversion:** Pressing the Up button moved the car backward. We adjusted the directional pin control assignments in software to correct forward and reverse polarity.
- **Left/Right Inversion:** Pressing Left caused the robot to turn right, and pressing Right caused it to turn left. Because the left and right drive motors were mapped to the opposite software channels, applying asymmetric turning PWM rotated the chassis in the reverse yaw direction.
- **Final Calibration:** We updated the pin mapping in the firmware so that the designated left drive channel controlled the physical left motor and the right drive channel controlled the physical right motor. All four directional movements were verified and fully synchronized with the mobile app interface.

5.1 end_of_the_day

1. Updated firmware to match ESP32 Core 3.x LEDC PWM architecture.
2. Established reliable BLE wireless pairing with the Dabble mobile application.
3. Calibrated directional polarities and fixed steering inversion for accurate mobile control.

6 Experience

Working on Part 1 of the UGV project was a great hands-on learning experience that taught us how hardware, software, and power supplies come together in robotics.

Hardware and Power Management

- **Dealing with Motor Current Spikes:** We learned why motors cannot be powered from the same source as sensitive microcontrollers. Powering the motors directly from the laptop USB port caused the ESP32 to brown out and reboot. Splitting the power into a 2S battery pack for the motors and a separate power bank for the ESP32 gave us a reliable and stable vehicle.
- **Understanding the L293D Motor Driver:** It was interesting to see how the L293D H-bridge works as an electronic switch. Controlling the direction by setting one input HIGH and the other LOW made the concept of differential drive very clear.

Embedded Software and Troubleshooting

- **Fixing Serial Upload Issues:** Getting the "Write timeout" error was frustrating at first, but checking Device Manager helped us find that the CP2102 driver was missing. Installing the driver and lowering the baud rate fixed the issue and taught us how to troubleshoot computer-to-board communication.
- **Updating Code for New Libraries:** Encountering errors with `ledcSetup` showed us that software libraries change over time. Updating our sketch to the newer ESP32 Core 3.0 API was straightforward once we understood the updated syntax.
- **Calibrating Motor Directions:** Fixing inverted controls in software rather than rewiring components was very convenient. Adjusting the code step-by-step until the car moved correctly in all directions helped us understand how differential steering works.

7 Firmware Implementation

The final Arduino firmware used for wireless BLE remote control with the Dabble application is provided below:

```
#define CUSTOM_SETTINGS
#define INCLUDE_GAMEPAD_MODULE
#include <DabbleESP32.h>

// L293D Motor Control Pins (Calibrated Channels)
const int LEFT_MOTOR_IN1  = 21;
const int LEFT_MOTOR_IN2  = 22;
const int RIGHT_MOTOR_IN3 = 18;
const int RIGHT_MOTOR_IN4 = 19;

// PWM Configuration (ESP32 Core 3.x API)
const int PWM_FREQ       = 1000; // 1 kHz switching frequency
const int PWM_RESOLUTION = 8;    // 8-bit duty resolution (0-255)

// Vehicle Base Speed
int baseSpeed = 220;

void setMotorPWM(int leftSpeed, int rightSpeed) {
  // Left Motor Control
  if (leftSpeed > 0) {
    ledcWrite(LEFT_MOTOR_IN1, constrain(leftSpeed, 0, 255));
    ledcWrite(LEFT_MOTOR_IN2, 0);
  } else if (leftSpeed < 0) {
    ledcWrite(LEFT_MOTOR_IN1, 0);
    ledcWrite(LEFT_MOTOR_IN2, constrain(abs(leftSpeed), 0, 255));
  } else {
    ledcWrite(LEFT_MOTOR_IN1, 0);
    ledcWrite(LEFT_MOTOR_IN2, 0);
  }

  // Right Motor Control
  if (rightSpeed > 0) {
    ledcWrite(RIGHT_MOTOR_IN3, constrain(rightSpeed, 0, 255));
    ledcWrite(RIGHT_MOTOR_IN4, 0);
  } else if (rightSpeed < 0) {
    ledcWrite(RIGHT_MOTOR_IN3, 0);
    ledcWrite(RIGHT_MOTOR_IN4, constrain(abs(rightSpeed), 0, 255));
  } else {
    ledcWrite(RIGHT_MOTOR_IN3, 0);
    ledcWrite(RIGHT_MOTOR_IN4, 0);
  }
```

```
}

void setup() {
  Serial.begin(115200);

  // Initialize Dabble BLE broadcast
  Dabble.begin("UGV-ESP32");

  // Attach GPIOs directly using Core 3.x API
  ledcAttach(LEFT_MOTOR_IN1,  PWM_FREQ, PWM_RESOLUTION);
  ledcAttach(LEFT_MOTOR_IN2,  PWM_FREQ, PWM_RESOLUTION);
  ledcAttach(RIGHT_MOTOR_IN3, PWM_FREQ, PWM_RESOLUTION);
  ledcAttach(RIGHT_MOTOR_IN4, PWM_FREQ, PWM_RESOLUTION);

  // Initialize motors to stopped state
  setMotorPWM(0, 0);
  Serial.println("UGV BLE Ready.");
}

void loop() {
  // Read incoming BLE control packets
  Dabble.processInput();

  // Digital D-Pad Directional Control
  if (GamePad.isUpPressed()) {
    setMotorPWM(baseSpeed, baseSpeed);                 // Forward
  }
  else if (GamePad.isDownPressed()) {
    setMotorPWM(-baseSpeed, -baseSpeed);               // Reverse
  }
  else if (GamePad.isLeftPressed()) {
    setMotorPWM(-baseSpeed * 0.75, baseSpeed * 0.75); // Turn Left
  }
  else if (GamePad.isRightPressed()) {
    setMotorPWM(baseSpeed * 0.75, -baseSpeed * 0.75); // Turn Right
  }
  else {
    setMotorPWM(0, 0);                                 // Stop
  }

  // Speed Adjustment Controls
  if (GamePad.isTrianglePressed()) {
    baseSpeed = min(255, baseSpeed + 20);
    delay(150);
  }
  if (GamePad.isCrossPressed()) {
    baseSpeed = max(100, baseSpeed - 20);
    delay(150);
  }
}
```